

POLITÉCNICO COLOMBIANO
JAIME ISAZA CADAVID

GUÍA PRÁCTICA 1

VISIÓN POR COMPUTADOR

Informe de solución de los cinco retos

Programa académico: Ingeniería Informática

Asignatura: Visión por computador e Inteligencia Artificial

Código: ING001547

Estudiante: Nicholas Pareja / Brayhan Escobar / Daniel Ospina

Fecha de entrega: 25 de agosto de 2026

Introducción

La presente guía práctica integra cinco retos orientados a repasar y aplicar conceptos fundamentales de visión por computador: representación de imágenes mediante matrices, convolución, análisis de color, segmentación y seguimiento de objetos, y filtrado de ruido. El desarrollo se realizó con Python, OpenCV, NumPy, Matplotlib, Google Colab y una hoja de cálculo compatible con Microsoft Excel, de acuerdo con las restricciones específicas establecidas para cada reto.

El informe reúne en un solo documento los fragmentos de código más relevantes, resultados obtenidos, evidencias visuales, análisis y conclusiones. En los retos que prohíben el uso inicial de librerías especializadas se implementaron explícitamente los cálculos o filtros solicitados antes de compararlos con soluciones optimizadas.

Resumen de cumplimiento

Reto	Peso	Resultado principal
1. Matrices a imágenes	20 %	Matriz 1000×1000, estadísticas manuales, archivo TXT, Colab/NumPy y visualización.
2. Convoluciones en Excel	10 %	Matriz 30×30, figura no rectangular, cuatro kernels y comparación visual.
3. Colores	10 %	Seis ROI, medias y desviaciones BGR, recuadros y análisis por color.
4. Seguimiento de figura en video	30 %	Segmentación HSV de rojo, verde y azul con bounding box y seguimiento.
5. Convoluciones para filtrado	30 %	Ruido Salt & Pepper, media/Gauss/mediana manual y comparación con medianBlur.

Metodología general

- Entorno local: Python 3.12.5 en Visual Studio Code con entorno virtual.
- Notebooks: Jupyter para los retos 1, 3 y 5; Google Colab para la segunda fase del reto 1.
- Visión por computador: OpenCV para lectura, conversión de espacios de color, captura de cámara y operaciones permitidas.
- Hoja de cálculo: matriz 30×30 con fórmulas y formato condicional, exportada finalmente como archivo .xlsx.
- Evidencias: salidas de notebooks, capturas del seguimiento por cámara y resultados visuales de los filtros.

1. Reto “Matrices a imágenes”

1.1 Objetivo

Construir una matriz de 1000×1000 valores aleatorios entre 0 y 255 sin utilizar NumPy para la creación ni para las primeras estadísticas; calcular mínimo, máximo, media y desviación estándar mediante fórmulas propias; medir tiempos, aplanar y almacenar los datos, leerlos posteriormente desde Google Drive en Colab con NumPy y visualizar la matriz como imagen.

1.2 Cumplimiento de requisitos

Requisito de la guía	Evidencia desarrollada
Matriz 1000×1000 sin NumPy	Lista de listas con <code>random.randint(0, 255)</code> .
Estadísticas manuales	Función propia para mínimo, máximo, media, varianza y desviación estándar.
Medición de tiempo	<code>time.perf_counter()</code> desde creación hasta estadísticas.
Aplanamiento y archivo	Vector de 1.000.000 de elementos guardado en <code>vector_matriz.txt</code> .
Google Drive + Colab	Archivo leído desde Drive con <code>np.loadtxt(..., dtype=np.uint8)</code> .
Comparación y visualización	Estadísticas con NumPy, segundo tiempo y visualización con Matplotlib.

1.3 Implementación manual

Creación de la matriz sin NumPy

```
import random
import time
import math

matriz = []
for i in range(1000):
    fila = []
    for j in range(1000):
        fila.append(random.randint(0, 255))
    matriz.append(fila)
```

Función de estadísticas implementada con fórmulas matemáticas

```
def calcular_estadisticas(matriz):
    minimo, maximo, suma, cantidad = 255, 0, 0, 0
    for fila in matriz:
        for valor in fila:
            minimo = min(minimo, valor)
            maximo = max(maximo, valor)
            suma += valor
            cantidad += 1

    media = suma / cantidad
    suma_diferencias = 0
    for fila in matriz:
        for valor in fila:
            suma_diferencias += (valor - media) ** 2

    varianza = suma_diferencias / cantidad
```

```
desviacion = math.sqrt(varianza)
return minimo, maximo, media, desviacion
```

La desviación estándar se calculó como raíz cuadrada de la varianza poblacional. Debido a que primero es necesario conocer la media, la matriz se recorre una segunda vez para acumular las diferencias cuadráticas respecto a esa media.

1.4 Aplanamiento, almacenamiento y Colab

Aplanamiento y almacenamiento local

```
vector = []
for fila in matriz:
    for valor in fila:
        vector.append(valor)

with open('datos/vector_matriz.txt', 'w') as archivo:
    for valor in vector:
        archivo.write(str(valor) + '\n')
```

Lectura y procesamiento en Google Colab con NumPy

```
vector_numpy = np.loadtxt(ruta, dtype=np.uint8)
minimo_numpy = np.min(vector_numpy)
maximo_numpy = np.max(vector_numpy)
media_numpy = np.mean(vector_numpy)
desviacion_numpy = np.std(vector_numpy)
imagen = vector_numpy.reshape(1000, 1000)
```

1.5 Resultados

Parámetro	Python manual	NumPy en Colab
Cantidad de valores	1.000.000	1.000.000
Mínimo	0	0
Máximo	255	255
Media	127.471889	127.471889
Desviación estándar	73.93894160568772	73.93894160570383
Tiempo registrado	0.6441 s	0.3992 s

Imagen generada a partir de la matriz aleatoria

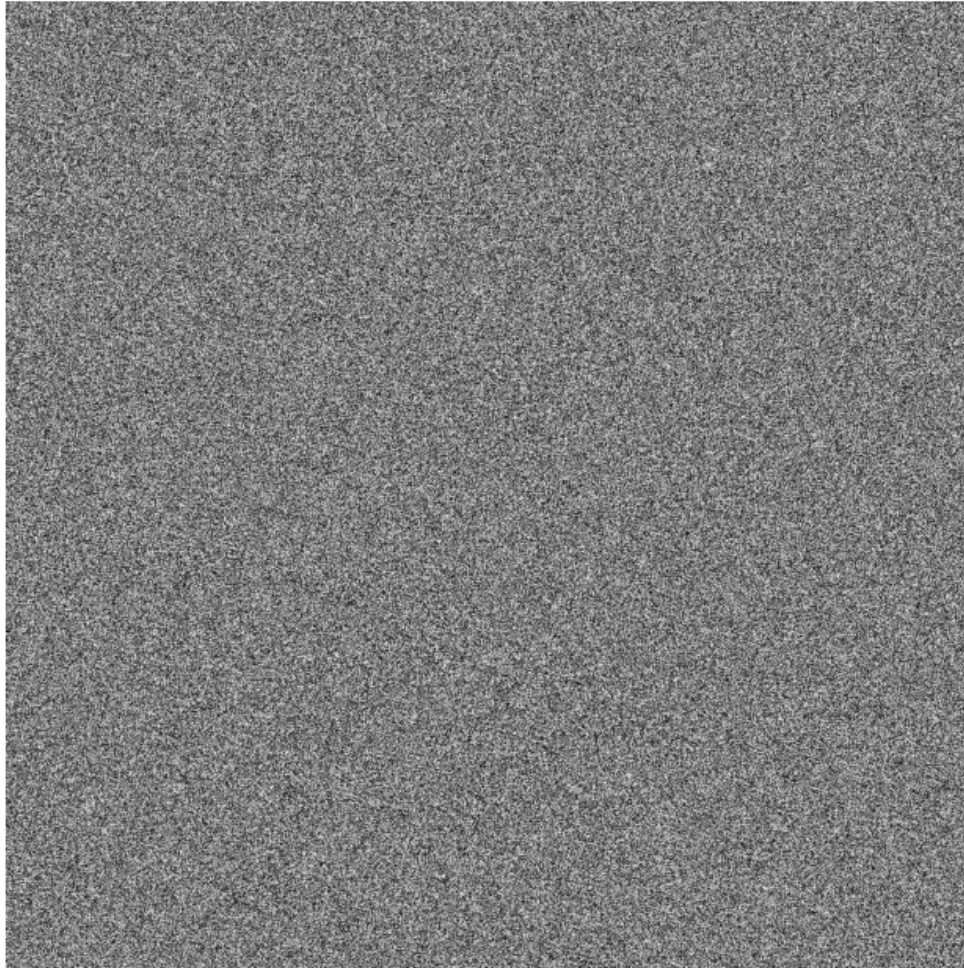


Figura 1. Visualización de la matriz aleatoria 1000×1000 como imagen en escala de grises.

1.6 Análisis

Los valores mínimo y máximo coinciden con los extremos permitidos (0 y 255). La media de 127.471889 se encuentra muy próxima a 127.5, el punto medio del intervalo, lo cual es consistente con la generación uniforme de una muestra muy grande. La desviación estándar, aproximadamente 73.94, también corresponde al comportamiento esperado de una distribución uniforme discreta en ese rango.

La imagen visualizada presenta apariencia de estática porque cada píxel fue generado de manera independiente y no existe estructura espacial entre vecinos. Los resultados manuales y los calculados con NumPy son equivalentes; la mínima diferencia en los últimos decimales corresponde a precisión de punto flotante. En la prueba realizada, el flujo con NumPy registró menor tiempo. Las dos mediciones incluyen operaciones diferentes de entrada/salida, por lo que se interpretan como comparación experimental de los flujos implementados y no como un benchmark aislado de cada biblioteca.

1.7 Conclusiones

1. Una imagen en escala de grises puede representarse mediante una matriz cuyos valores de 0 a 255 corresponden a intensidades de píxel.

- La implementación manual permitió comprobar matemáticamente el cálculo de media y desviación estándar sin depender de librerías matriciales.
- El aplanamiento no elimina información: únicamente cambia la organización de 1000×1000 a un vector de 1.000.000 de elementos.
- NumPy reprodujo las estadísticas obtenidas manualmente y presentó una ejecución más rápida en la prueba realizada.
- La matriz aleatoria genera una imagen tipo ruido por la ausencia de relaciones espaciales entre píxeles.

2. Reto “Convoluciones en Excel”

2.1 Objetivo

Representar una imagen binaria en una matriz de 30×30 celdas, utilizar 0 para negro y 255 para blanco mediante formato condicional, aplicar diferentes kernels de convolución y comparar visualmente sus efectos. El trabajo se construyó en una hoja de cálculo y se exportó como archivo compatible con Microsoft Excel (.xlsx).

2.2 Matriz y figura interna

La matriz se creó en el rango A1:AD30. El fondo fue inicializado en 255 y la figura interna en 0. Se eligió un corazón pixelado para cumplir la condición de utilizar una forma distinta de un rectángulo o cuadrado.

1	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	AA	AB	AC	AD	AE
2		255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255
3		255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255
4		255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255
5		255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255
6		255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255
7		255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255
8		255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255
9		255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255
10		255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255
11		255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255
12		255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255
13		255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255
14		255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255
15		255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255
16		255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255
17		255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255
18		255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255
19		255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255
20		255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255
21		255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255
22		255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255
23		255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255
24		255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255
25		255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255
26		255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255
27		255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255
28		255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255
29		255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255
30		255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255

Figura 2. Matriz 30×30 con corazón negro (0) sobre fondo blanco (255).

2.3 Kernels probados

Kernel	Matriz	Finalidad
Identidad	0 0 0 / 0 1 0 / 0 0 0	Conservar el píxel central y verificar el procedimiento.
Bordes (8 vecinos)	-1 -1 -1 / -1 8 -1 / -1 -1 -1	Resaltar cambios fuertes de intensidad.
Media 3×3	1/9 en las nueve posiciones	Suavizar mediante promedio local.
Realce definido para la práctica	0 -1 0 / -1 5 -1 / 0 -1 0	Aumentar el peso del centro y realzar detalles.

La convolución se implementó con SUMAPRODUCTO. Por ejemplo, para el kernel identidad se utilizó una ventana 3×3 de la imagen y el kernel fijado mediante referencias absolutas:

Fórmula base de convolución en la hoja de cálculo

=SUMAPRODUCTO(Imagen_Original!A1:C3;\$AF\$2:\$AH\$4)

Al desplazar la fórmula por la hoja, la ventana sobre la imagen cambia mediante referencias relativas, mientras que las celdas del kernel permanecen fijas gracias a los signos \$. La salida válida para una imagen 30×30 con un kernel 3×3 es de 28×28.

2.4 Resultados visuales

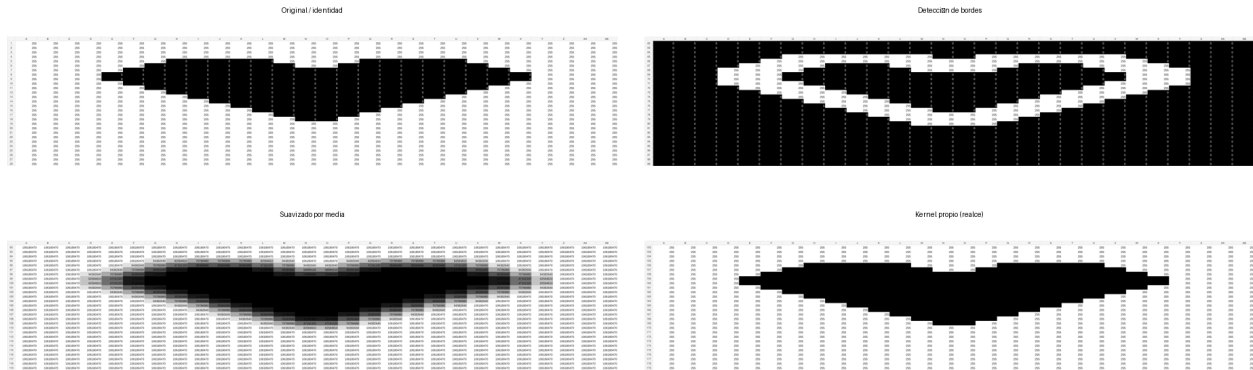


Figura 3. Comparación visual de identidad, detección de bordes, suavizado por media y realce.

2.5 Análisis

El kernel identidad conservó la forma del corazón, confirmando que la fórmula y el desplazamiento del vecindario funcionaban correctamente. El kernel de bordes respondió principalmente en las transiciones entre 0 y 255, por lo que el contorno de la figura se hizo visible mientras las regiones uniformes tendieron a cero.

El kernel de media generó valores intermedios alrededor del límite de la figura. Esos valores aparecen como grises y producen un efecto de suavizado. Por su parte, el kernel de realce asignó mayor peso al píxel central y valores negativos a sus vecinos, aumentando el contraste alrededor de las transiciones.

2.6 Conclusiones

1. Una imagen puede modelarse directamente con celdas numéricas y visualizarse mediante formato condicional.
2. La convolución depende de los coeficientes del kernel y del vecindario de píxeles sobre el cual se aplica.
3. Los kernels de detección de bordes responden a cambios de intensidad, mientras los de promedio reducen variaciones locales.
4. El uso de varios kernels permitió observar de forma práctica que modificar los pesos produce transformaciones visuales distintas.

3. Reto “Colores”

3.1 Objetivo

Analizar cuantitativamente el color de seis regiones de una imagen: cinco regiones relativamente homogéneas de colores diferentes y una región heterogénea. Para cada ROI se calcularon la media y la desviación estándar de los tres canales de la imagen leída con OpenCV.

3.2 Lectura y regiones de interés

La imagen se cargó con OpenCV y presentó shape (455, 728, 3), correspondiente a 455 píxeles de alto, 728 de ancho y tres canales. OpenCV organiza por defecto la imagen a color en BGR, por lo que el canal 0 es azul, el 1 verde y el 2 rojo.

Carga de imagen y coordenadas de las ROI

```
imagen = cv2.imread('imagenes/imagen_colores.jpg')
print(imagen.shape) # (455, 728, 3)

regiones = {
    'Cielo azul': (60, 25, 100, 65),
    'Nieve blanca': (318, 58, 350, 88),
    'Verde oscuro': (438, 92, 470, 124),
    'Verde claro': (30, 355, 62, 387),
    'Naranja / rocas': (438, 385, 470, 417),
    'Mezcla de colores': (285, 330, 345, 390)
}
```

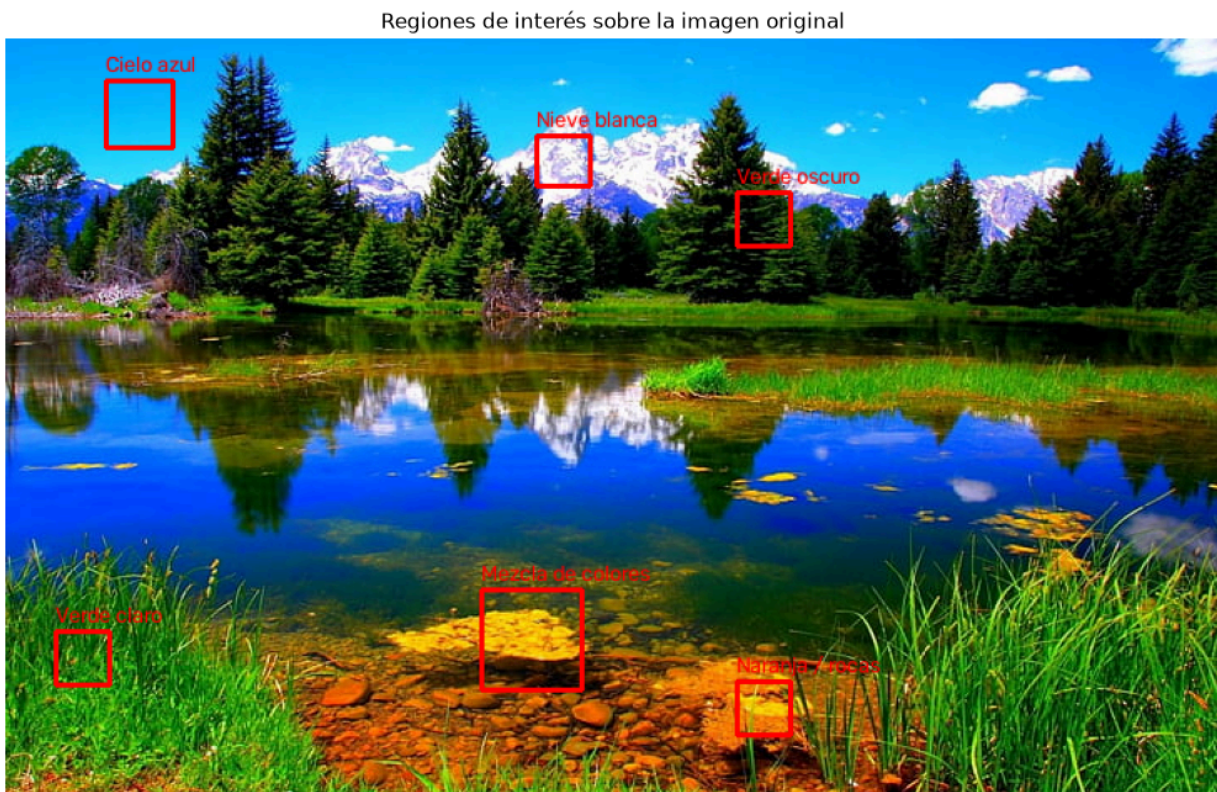


Figura 4. Las seis regiones de interés dibujadas sobre la imagen original.

3.3 Cálculo de estadísticas por canal

Cálculo de media y desviación estándar BGR

```
roi = imagen[y1:y2, x1:x2]
media_b = np.mean(roi[:, :, 0])
media_g = np.mean(roi[:, :, 1])
media_r = np.mean(roi[:, :, 2])
std_b = np.std(roi[:, :, 0])
std_g = np.std(roi[:, :, 1])
std_r = np.std(roi[:, :, 2])
```

Región	Media B	Media G	Media R	Std B	Std G	Std R
Cielo azul	253.78	190.02	10.49	0.94	10.11	7.91
Nieve blanca	239.28	212.26	213.07	16.79	37.34	43.32
Verde oscuro	15.94	35.30	21.39	24.95	34.25	26.82
Verde claro	19.44	119.92	41.53	22.65	37.16	36.76
Naranja / rocas	14.61	140.50	224.48	14.84	44.42	36.60
Mezcla de colores	16.13	105.86	140.09	15.09	68.93	87.31

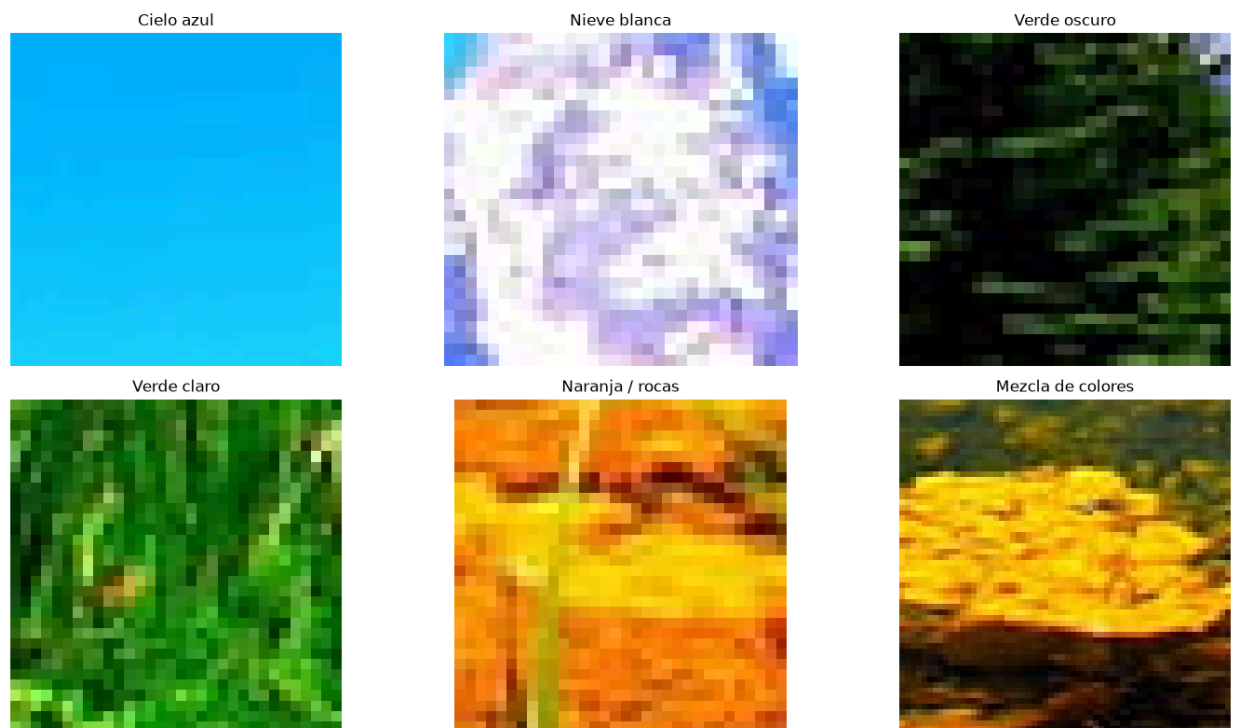


Figura 5. Visualización individual de las seis regiones seleccionadas.

3.4 Análisis

En el cielo azul, $B = 253.78$ es el canal dominante y $R = 10.49$ es muy bajo, lo que coincide con su apariencia. Además, la desviación del canal azul es apenas 0.94, indicando una zona altamente homogénea. En la nieve, los tres canales presentan valores altos, propios de una región cercana al blanco, aunque con desviaciones mayores por sombras y relieve.

En el verde oscuro los tres promedios son bajos, pero G es el mayor. En el verde claro, G aumenta hasta 119.92 y domina con claridad. Para las rocas naranjas, R = 224.48 y G = 140.50 son mucho mayores que B = 14.61, combinación coherente con el tono naranja.

La región de mezcla presenta desviaciones especialmente altas en verde (68.93) y rojo (87.31), lo que confirma una mayor heterogeneidad. La desviación estándar sirve así como indicador cuantitativo de la variabilidad interna de una región.

3.5 Conclusiones

1. Los valores medios de los canales BGR permiten relacionar numéricamente una región con su color aparente.
2. Las zonas claras presentan valores altos en varios canales, mientras las zonas oscuras concentran valores más bajos.
3. La desviación estándar permite diferenciar regiones relativamente uniformes de regiones con mezcla de intensidades y colores.
4. El análisis localizado mediante ROI permite estudiar la información cromática sin procesar toda la imagen como una única población.

4. Reto “Seguimiento de figura en video”

4.1 Objetivo

Capturar video desde la cámara, segmentar una figura de color sobre fondo blanco, localizarla mediante un bounding box y realizar seguimiento cuadro a cuadro. El algoritmo debía clasificar tres colores diferentes y escribir el nombre correspondiente sobre la imagen original.

4.2 Flujo del algoritmo

- Captura de cada frame con `cv2.VideoCapture(0)`.
- Conversión del frame desde BGR a HSV.
- Creación de máscaras binarias con `cv2.inRange` para rojo, verde y azul.
- Búsqueda de contornos externos y descarte de regiones con área menor o igual a 500 píxeles.
- Cálculo del rectángulo delimitador con `cv2.boundingRect`.
- Dibujo del bounding box y texto ROJO, VERDE o AZUL sobre el frame.
- Repetición del procedimiento para cada cuadro del video, produciendo el seguimiento.

4.3 Rangos HSV utilizados

Color	Rango HSV inferior	Rango HSV superior
Rojo 1	[0, 120, 70]	[10, 255, 255]
Rojo 2	[170, 120, 70]	[180, 255, 255]
Verde	[35, 80, 50]	[85, 255, 255]
Azul	[90, 80, 50]	[130, 255, 255]

Se emplearon dos rangos para el rojo porque el tono rojo se encuentra en los extremos del eje de Hue de OpenCV. Las máscaras de ambos intervalos se combinaron con `bitwise_or`.

Estructura de segmentación, contorno y bounding box

```
hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
mascara = cv2.inRange(hsv, limite_inferior, limite_superior)
contornos, _ = cv2.findContours(
    mascara, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

for contorno in contornos:
    if cv2.contourArea(contorno) > 500:
        x, y, w, h = cv2.boundingRect(contorno)
        cv2.rectangle(frame, (x, y), (x+w, y+h), color_bgr, 2)
        cv2.putText(frame, nombre_color, (x, y-10),
            cv2.FONT_HERSHEY_SIMPLEX, 0.8, color_bgr, 2)
```

4.4 Evidencias

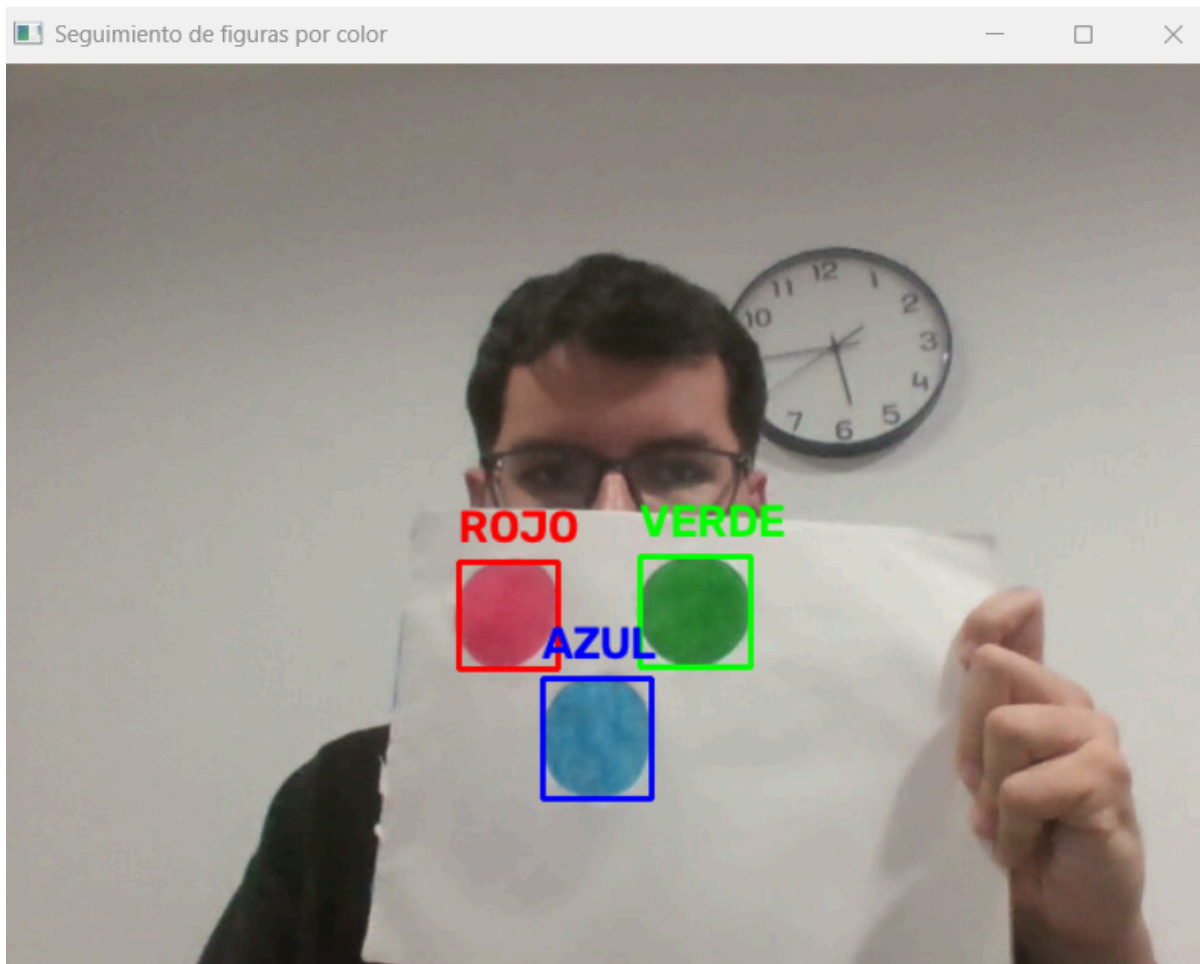


Figura 6. Detección simultánea de los tres colores con su etiqueta y bounding box.

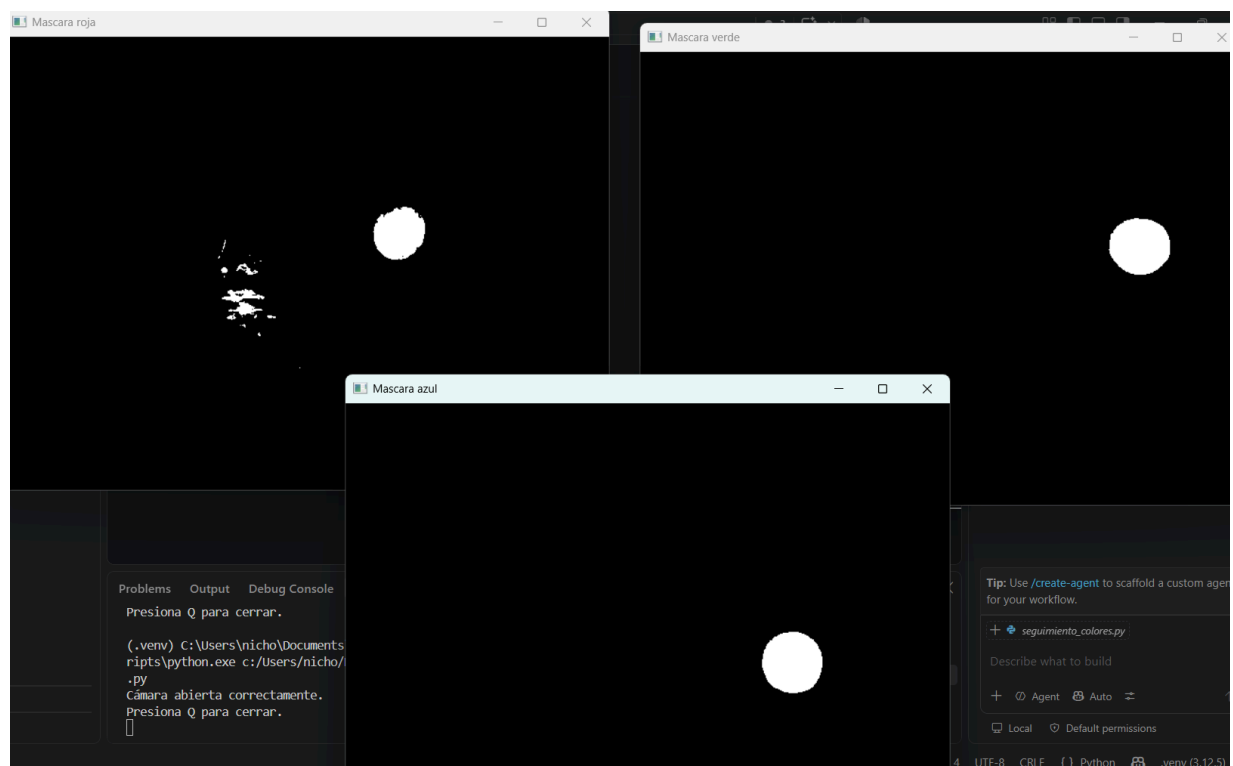


Figura 7. Máscaras binarias de rojo, verde y azul; los píxeles detectados aparecen en blanco.

4.5 Análisis

El algoritmo identificó correctamente las figuras roja, verde y azul y actualizó la posición del bounding box mientras se desplazaban frente a la cámara. La representación HSV facilitó la segmentación porque separa el tono de la intensidad y la saturación, lo que permite definir intervalos de color más interpretables que en BGR.

Las máscaras muestran en blanco las zonas que satisfacen los rangos HSV y en negro el resto. A partir de esas regiones se obtienen los contornos y posteriormente el rectángulo delimitador. El umbral de área de 500 píxeles evita que pequeñas detecciones espurias generen cajas de seguimiento.

4.6 Conclusiones

1. La segmentación HSV permitió clasificar correctamente tres colores diferentes en tiempo real.
2. Las máscaras binarias constituyen una etapa intermedia útil para separar el objeto del fondo.
3. La combinación de contornos y bounding boxes permite localizar el objeto y actualizar su posición cuadro a cuadro.
4. La iluminación del entorno influye en los valores HSV; por ello, los rangos pueden requerir ajustes cuando cambian las condiciones de captura.

5. Reto “Convoluciones para filtrado”

5.1 Objetivo

Partir de una imagen limpia, añadir ruido Salt & Pepper y probar diferentes estrategias para reducirlo. Las primeras operaciones de filtrado se implementaron manualmente, sin utilizar funciones de filtrado de OpenCV, y posteriormente se comparó la mediana propia con cv2.medianBlur().

5.2 Generación del ruido

Ruido Salt & Pepper agregado manualmente al 5 % de los píxeles

```
def agregar_ruido_salt_pepper(imagen, porcentaje=0.05):
    imagen_ruido = imagen.copy()
    alto, ancho, canales = imagen_ruido.shape
    cantidad = int(alto * ancho * porcentaje)

    for _ in range(cantidad):
        x = random.randint(0, ancho - 1)
        y = random.randint(0, alto - 1)
        if random.random() < 0.5:
            imagen_ruido[y, x] = [255, 255, 255]
        else:
            imagen_ruido[y, x] = [0, 0, 0]
    return imagen_ruido
```

5.3 Convolución manual: media y Gaussiano

Función de convolución implementada sin funciones de filtrado de OpenCV

```
def convolucion_manual(imagen, kernel):
    alto, ancho, canales = imagen.shape
    k_alto, k_ancho = kernel.shape
    my, mx = k_alto // 2, k_ancho // 2
    padding = np.pad(imagen, ((my,my),(mx,mx),(0,0)), mode='edge')
    resultado = np.zeros_like(imagen, dtype=np.float64)

    for y in range(alto):
        for x in range(ancho):
            region = padding[y:y+k_alto, x:x+k_ancho]
            for canal in range(canales):
                resultado[y,x,canal] = np.sum(region[:, :, canal] * kernel)

    return np.clip(resultado, 0, 255).astype(np.uint8)
```

Filtro	Kernel 3×3
Media	1/9 en las nueve posiciones
Gaussiano	$(1/16) \times \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$

5.4 Mediana manual y comparación con OpenCV

Filtro de mediana propio y llamada de comparación con OpenCV

```
def filtro_mediana_manual(imagen, tamano=3):
    alto, ancho, canales = imagen.shape
    margen = tamano // 2
    padding = np.pad(imagen,
        ((margen,margen),(margen,margen),(0,0)),
```



```

        mode='edge')
resultado = np.zeros_like(imagen)

for y in range(alto):
    for x in range(ancho):
        region = padding[y:y+tamano, x:x+tamano]
        for canal in range(canales):
            valores = sorted(region[:, :, canal].flatten())
            resultado[y,x,canal] = valores[len(valores)//2]
    return resultado

imagen_mediana_opencv = cv2.medianBlur(imagen_ruido, 3)

```

5.5 Resultados



Figura 8. Comparación visual: imagen original, ruido, media manual, Gaussiano manual, mediana manual y mediana OpenCV.

Método	Observación
Media manual	Reduce parte del ruido, pero introduce mayor desenfoque.
Gaussiano manual	Suaviza de forma ponderada; todavía quedan impulsos de ruido.
Mediana manual	Elimina mejor puntos extremos y conserva mejor los bordes.
cv2.medianBlur()	Resultado visual muy similar a la mediana manual, con ejecución mucho más rápida.

Tiempo medido	Resultado
Filtro mediana manual	2.5798624999588355 s
cv2.medianBlur()	0.0004152000183239579 s

5.6 Análisis

El ruido Salt & Pepper introduce píxeles extremos de 0 y 255. El filtro de media reduce parte de esos impulsos, pero como los valores extremos participan directamente en el promedio, el resultado pierde nitidez. El filtro Gaussiano pondera más el centro del vecindario y genera un suavizado más controlado, aunque sigue siendo un filtro lineal sensible a impulsos fuertes.

La mediana obtuvo el mejor comportamiento visual para este tipo de ruido porque selecciona el valor central de los valores ordenados y, por tanto, es menos sensible a observaciones extremas. La versión manual y `cv2.medianBlur()` produjeron resultados similares con ventana 3×3, pero OpenCV fue notablemente más rápido: 2.5799 s frente a 0.000415 s en la prueba registrada.

5.7 Conclusiones

1. El ruido Salt & Pepper puede simularse sustituyendo aleatoriamente píxeles por negro o blanco.
2. Los filtros lineales de media y Gaussiano reducen ruido, pero también suavizan información útil de la imagen.
3. La mediana es especialmente apropiada para ruido impulsivo porque limita la influencia de valores extremos.
4. La implementación manual permite comprender el algoritmo, mientras que OpenCV aporta una optimización de tiempo muy superior para aplicaciones prácticas.

Conclusiones generales

1. Las imágenes digitales pueden analizarse como estructuras matriciales, lo que permite aplicar estadística, álgebra y operaciones locales sobre píxeles.
2. Las convoluciones muestran cómo una pequeña matriz de pesos puede producir efectos tan diferentes como conservación, suavizado, detección de bordes o realce.
3. El análisis por canales y regiones de interés convierte la apariencia visual del color en variables cuantitativas como media y desviación estándar.
4. La segmentación por color en HSV, combinada con contornos y bounding boxes, permite desarrollar sistemas sencillos de detección y seguimiento en tiempo real.
5. La elección del filtro debe responder al tipo de ruido: para Salt & Pepper, la mediana produjo el resultado más adecuado entre los métodos probados.
6. Implementar algoritmos manualmente facilita comprender su fundamento, mientras que librerías como NumPy y OpenCV ofrecen optimizaciones necesarias cuando el volumen de datos o la velocidad de procesamiento son importantes.

Referencias

- Vásquez, R. (2026). Guía 1: Visión por Computador. Curso Visión por Computador e IA. Politécnico Colombiano Jaime Isaza Cadavid.
- Howse, J., & Minichino, J. (2020). Learning OpenCV 4 Computer Vision with Python 3 (3.^a ed.). Packt Publishing.
- Szeliski, R. (2010). Computer Vision: Algorithms and Applications. Springer.
- González, R. C., Woods, R. E., & Eddins, S. L. (2004). Tratamiento digital de imágenes. Addison-Wesley.

OpenCV. Thresholding Operations using inRange.

https://docs.opencv.org/4.x/da/d97/tutorial_threshold_inRange.html

OpenCV. Laplace Operator. https://docs.opencv.org/4.x/d5/db5/tutorial_laplace_operator.html

OpenCV. Smoothing Images / Image Filtering. https://docs.opencv.org/4.x/d4/d13/tutorial_py_filtering.html